Complete Web Development Guide - HTML, CSS, XML & XHTML

UNIT-I: HTML (HyperText Markup Language)

What is HTML?

HTML is like the skeleton of a webpage. It tells the browser what content to display and how to structure it. Think of it as the blueprint of a house - it defines where everything goes.

Basic Syntax

HTML uses **tags** to mark up content. Tags are keywords surrounded by angle brackets:

```html
<tagname>Content goes here</tagname>
```

Key Rules:

- Most tags come in pairs: opening `<tag>` and closing `</tag>`
- Closing tags have a forward slash `/`
- Some tags are self-closing: `<br>`, `<img>`, `<hr>`

Standard HTML Document Structure

Every HTML page follows this basic structure:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
</head>
<body>
    <!-- Your content goes here -->
</body>
</html>
```

Breakdown:

- `<!DOCTYPE html>` - Tells browser this is HTML5
- `<html>` - Root element containing everything
- `<head>` - Contains information about the page (not visible to users)
- `<title>` - Shows in browser tab

- `<body>` - Contains all visible content

Basic Text Markup

Common Text Tags:

- `<h1>` to `<h6>` - Headings (largest to smallest)
- `<p>` - Paragraphs
- `<br>` - Line break
- `<strong>` or `<b>` - Bold text
- `<em>` or `<i>` - Italic text
- `<u>` - Underlined text
- `<small>` - Smaller text
- `<mark>` - Highlighted text

Example:

```html
<h1>Main Heading</h1>
<p>This is a <strong>bold</strong> paragraph with <em>italic</em> text.</p>
```

HTML Styles

You can add basic styling directly to HTML elements using the `style` attribute:

```html
<p style="color: blue; font-size: 18px;">This is blue text</p>
```

Common Style Properties:

- `color` - Text color
- `background-color` - Background color
- `font-size` - Text size
- `font-family` - Font type
- `text-align` - Text alignment

Elements and Attributes

Elements are the building blocks (like `<p>`, `<h1>`, `<img>`)

Attributes provide additional information about elements:

```html
<img src="photo.jpg" alt="A beautiful sunset" width="300" height="200">
```

Common Attributes:

- `id` - Unique identifier
- `class` - Group identifier for styling
- `src` - Source file path
- `alt` - Alternative text
- `href` - Link destination
- `title` - Tooltip text

Headings

Six levels of headings for organizing content:

```html
<h1>Main Title</h1>
<h2>Chapter Title</h2>
<h3>Section Title</h3>
<h4>Subsection</h4>
<h5>Minor Heading</h5>
<h6>Smallest Heading</h6>
```

Layouts

HTML5 provides semantic elements for better page structure:

```html
<header>Top of page content</header>
<nav>Navigation menu</nav>
<main>
    <section>Main content section</section>
    <article>Individual article</article>
    <aside>Sidebar content</aside>
</main>
<footer>Bottom of page content</footer>
```

Iframes

Iframes embed other web pages within your page:

```html
<iframe src="https://www.example.com" width="600" height="400"></iframe>
```

Common Uses:

- Embedding YouTube videos
- Displaying maps
- Including external content

Images

Display pictures on your webpage:

```html
<img src="image.jpg" alt="Description" width="300" height="200">
```

Key Attributes:

- `src` - Image file path
- `alt` - Description for screen readers
- `width/height` - Size control
- `title` - Hover tooltip

Hypertext Links

Create clickable links:

```html
<a href="https://www.example.com">Visit Example</a>
<a href="page2.html">Go to Page 2</a>
<a href="#section1">Jump to Section 1</a>
<a href="mailto:someone@example.com">Send Email</a>
```

Link Types:

- External links (other websites)
- Internal links (same website)
- Anchor links (same page)
- Email links

Lists

Unordered Lists (bullets):

```html
<ul>
    <li>First item</li>
    <li>Second item</li>
    <li>Third item</li>
</ul>
```

Ordered Lists (numbers):

```html
<ol>
    <li>First step</li>
    <li>Second step</li>
    <li>Third step</li>
</ol>
```

Description Lists:

```html
<dl>
    <dt>Term</dt>
    <dd>Definition</dd>
</dl>
```

Tables

Display data in rows and columns:

```html
<table>
    <thead>
        <tr>
            <th>Name</th>
            <th>Age</th>
            <th>City</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>John</td>
            <td>25</td>
            <td>New York</td>
        </tr>
        <tr>
            <td>Jane</td>
            <td>30</td>
            <td>London</td>
        </tr>
    </tbody>
</table>
```

Table Elements:

- `<table>` - Container
- `<thead>` - Header section
- `<tbody>` - Body section
- `<tr>` - Table row
- `<th>` - Header cell
- `<td>` - Data cell

Forms

Collect user input:

```html
<form action="submit.php" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label for="message">Message:</label>
    <textarea id="message" name="message"></textarea>

    <input type="submit" value="Submit">
</form>
```

Form Input Types:

- `text` - Single line text
- `email` - Email address
- `password` - Hidden text
- `number` - Numbers only
- `date` - Date picker
- `radio` - Single choice
- `checkbox` - Multiple choices
- `file` - File upload

Dynamic HTML

Dynamic HTML combines HTML with CSS and JavaScript to create interactive content:

Examples:

- Dropdown menus
- Image slideshows
- Form validation
- Content that changes without page reload

CSS (Cascading Style Sheets)

Need for CSS

Problems CSS Solves:

- HTML alone creates plain, boring pages
- Mixing content with presentation makes code messy
- Hard to maintain consistent styling across multiple pages
- Limited design flexibility with HTML alone

Introduction to CSS

CSS is like the interior designer for your HTML house. While HTML provides structure, CSS makes it beautiful by controlling:

- Colors and fonts
- Layout and spacing
- Animations and effects
- Responsive design

Basic Syntax and Structure

CSS Rule Structure:

```
selector {
    property: value;
    property: value;
}
```

Example:

```
h1 {
    color: blue;
    font-size: 24px;
    text-align: center;
}
```

Parts Explained:

- `h1` - Selector (what to style)
- `color: blue;` - Declaration (how to style)
- `color` - Property
- `blue` - Value

Using CSS

Three Ways to Add CSS:

1. **Inline CSS** (directly in HTML):

```html
<p style="color: red;">Red text</p>
```

2. **Internal CSS** (in HTML head):

```html
<head>
    <style>
        p { color: red; }
    </style>
</head>
```

3. **External CSS** (separate file):

```html
<head>
    <link rel="stylesheet" href="styles.css">
</head>
```

Background Images, Colors, and Properties

Background Colors:

```css
body {
    background-color: lightblue;
}
```

Background Images:

```css
body {
    background-image: url('background.jpg');
    background-repeat: no-repeat;
    background-position: center;
    background-size: cover;
}
```

Background Properties:

- `background-color` - Solid color
- `background-image` - Image background
- `background-repeat` - How image repeats
- `background-position` - Image placement
- `background-size` - Image size

Manipulating Text

Text Properties:

```
css

p {
    color: navy;
    text-align: center;
    text-decoration: underline;
    text-transform: uppercase;
    line-height: 1.5;
    letter-spacing: 2px;
    word-spacing: 5px;
}
```

Text Styling Options:

- `color` - Text color
- `text-align` - Alignment (left, center, right, justify)
- `text-decoration` - Underline, overline, line-through
- `text-transform` - uppercase, lowercase, capitalize
- `line-height` - Space between lines
- `letter-spacing` - Space between letters
- `word-spacing` - Space between words

Using Fonts

Font Properties:

```css
h1 {
    font-family: Arial, sans-serif;
    font-size: 24px;
    font-weight: bold;
    font-style: italic;
}
```

Font Options:

- `font-family` - Font type
- `font-size` - Text size
- `font-weight` - Thickness (normal, bold, 100-900)
- `font-style` - Normal, italic, oblique

Web Safe Fonts:

- Arial, Helvetica, sans-serif
- Georgia, serif
- "Times New Roman", serif
- "Courier New", monospace

Borders

Border Properties:

```css
div {
    border: 2px solid black;
    border-radius: 10px;
}
```

Detailed Border Control:

```css
div {
    border-width: 2px;
    border-style: solid;
    border-color: red;

    /* Individual sides */
    border-top: 1px dotted blue;
    border-right: 2px dashed green;
    border-bottom: 3px solid yellow;
    border-left: 4px double purple;
}
```

Boxes, Margins, and Padding

The Box Model: Every HTML element is a box with:

- Content (the actual content)
- Padding (space inside the box)
- Border (line around the box)
- Margin (space outside the box)

```css
div {
    width: 200px;
    height: 100px;
    padding: 20px;
    border: 5px solid black;
    margin: 10px;
}
```

Margin and Padding Shortcuts:

```
/* All sides */
margin: 10px;

/* Top/bottom, left/right */
margin: 10px 20px;

/* Top, left/right, bottom */
margin: 10px 20px 15px;

/* Top, right, bottom, left */
margin: 10px 20px 15px 25px;
```

Lists Styling

List Properties:

```
ul {
    list-style-type: circle;
    list-style-position: inside;
    list-style-image: url('bullet.png');
}

ol {
    list-style-type: upper-roman;
}
```

Remove List Bullets:

```
ul {
    list-style: none;
}
```

Positioning Using CSS

Position Types:

1. **Static** (default):

```
div { position: static; }
```

2. **Relative** (relative to normal position):

```
css
div {
    position: relative;
    top: 10px;
    left: 20px;
}
```

3. **Absolute** (relative to nearest positioned parent):

```
css
div {
    position: absolute;
    top: 0;
    right: 0;
}
```

4. **Fixed** (relative to viewport):

```
css
div {
    position: fixed;
    bottom: 0;
    right: 0;
}
```

5. **Sticky** (switches between relative and fixed):

```
css
div {
    position: sticky;
    top: 0;
}
```

CSS2 Features

CSS2 introduced:

- Better positioning
- Media types for different devices
- Downloadable fonts

- Table layout properties
- Counters and generated content

The Box Model (Detailed)

Standard Box Model: Total width = width + padding + border + margin

Box-sizing Property:

```
css

/* Standard box model */
div {
    box-sizing: content-box;
}

/* Alternative box model */
div {
    box-sizing: border-box;
}
```

Working with XML

What is XML?

XML (eXtensible Markup Language) is like HTML but for storing and transporting data. While HTML displays data, XML describes data.

Key Differences from HTML:

- XML tags are custom-made
- XML is case-sensitive
- All XML tags must be properly closed
- XML focuses on data, not presentation

XML Example:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book id="1">
        <title>Learning XML</title>
        <author>John Doe</author>
        <price>29.99</price>
    </book>
</bookstore>
```

Document Type Definition (DTD)

DTD defines the structure and rules for an XML document. It's like a blueprint that says what elements are allowed.

Internal DTD:

```xml
<?xml version="1.0"?>
<!DOCTYPE bookstore [
    <!ELEMENT bookstore (book+)>
    <!ELEMENT book (title, author, price)>
    <!ELEMENT title (#PCDATA)>
    <!ELEMENT author (#PCDATA)>
    <!ELEMENT price (#PCDATA)>
]>
<bookstore>
    <!-- XML content here -->
</bookstore>
```

External DTD:

```xml
<?xml version="1.0"?>
<!DOCTYPE bookstore SYSTEM "bookstore.dtd">
<bookstore>
    <!-- XML content here -->
</bookstore>
```

XML Schemas

XML Schema is a more powerful alternative to DTD. It provides:

- Data type support

- Better validation
- Namespace support

XML Schema Example:

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="book">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="title" type="xs:string"/>
                <xs:element name="author" type="xs:string"/>
                <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

Document Object Model (DOM)

DOM is a way to represent XML/HTML documents as a tree structure in memory. It allows programs to access and modify the document.

DOM Tree Structure:

```
Document
├── Root Element
    ├── Element
    │   ├── Text
    │   └── Attribute
    └── Element
        └── Text
```

DOM Methods:

- `getElementById()` - Find element by ID
- `getElementsByTagName()` - Find elements by tag
- `createElement()` - Create new element
- `appendChild()` - Add child element

Parsers - DOM and SAX

DOM Parser:

- Loads entire document into memory

- Creates tree structure
- Good for small documents
- Allows random access to any part

SAX Parser (Simple API for XML):

- Reads document sequentially
- Event-driven (triggers events for each element)
- Memory efficient
- Good for large documents
- Only forward reading

When to Use Which:

- Use DOM for small files that need modification
- Use SAX for large files that only need reading

Introduction to XHTML

What is XHTML?

XHTML (eXtensible HyperText Markup Language) is HTML written according to XML rules. It's stricter than HTML.

Key Differences from HTML:

- All tags must be closed
- All tags must be lowercase
- Attribute values must be quoted
- All attributes must have values

HTML vs XHTML:

```html
<!-- HTML (more forgiving) -->
<p>This is a paragraph
<br>
<input type="text" name="username">

<!-- XHTML (strict) -->
<p>This is a paragraph</p>
<br />
<input type="text" name="username" />
```

XML in XHTML

XHTML follows XML syntax rules:

XHTML Document Structure:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <title>XHTML Page</title>
</head>
<body>
    <p>Content goes here</p>
</body>
</html>
```

Meta Tags

Meta tags provide information about the HTML document:

```html
<head>
    <!-- Character encoding -->
    <meta charset="UTF-8" />

    <!-- Page description -->
    <meta name="description" content="Page description here" />

    <!-- Keywords for search engines -->
    <meta name="keywords" content="HTML, CSS, XHTML" />

    <!-- Author information -->
    <meta name="author" content="Your Name" />

    <!-- Viewport for responsive design -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!-- Refresh page every 30 seconds -->
    <meta http-equiv="refresh" content="30" />
</head>
```

Character Entities

Special characters in HTML/XHTML:

Common Entities:

- `<` - Less than (<)
- `>` - Greater than (>)
- `&` - Ampersand (&)
- `"` - Quotation mark (")
- `'` - Apostrophe (')
- ` ` - Non-breaking space
- `©` - Copyright symbol (©)
- `®` - Registered trademark (®)

Example:

```html
<p>To display &lt;HTML&gt; tags, use &amp;lt; and &amp;gt;</p>
```

Frames and Frame Sets

Frames divide the browser window into multiple sections (mostly deprecated now):

Frameset Example:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Frames Example</title>
</head>
<frameset cols="25%,75%">
    <frame src="menu.html" name="menu" />
    <frame src="content.html" name="content" />
</frameset>
</html>
```

Frame Attributes:

- `src` - Source file
- `name` - Frame name for targeting
- `scrolling` - Enable/disable scrollbars
- `noresize` - Prevent resizing
- `frameborder` - Show/hide borders

Modern Alternative - Iframes:

```html
<iframe src="content.html" width="800" height="600"></iframe>
```

Summary

This guide covers the fundamentals of web development:

HTML provides the structure and content of web pages using elements and attributes.

CSS handles the presentation and styling, making pages visually appealing and responsive.

XML is used for data storage and transport with strict syntax rules.

XHTML combines HTML with XML's strict syntax for cleaner, more maintainable code.

These technologies work together to create modern web applications. HTML provides the foundation, CSS makes it beautiful, XML handles data, and XHTML ensures clean, valid markup.

Best Practices:

- Always use proper document structure
- Separate content (HTML) from presentation (CSS)
- Validate your markup
- Use semantic HTML elements
- Keep CSS organized and maintainable
- Follow accessibility guidelines
- Test across different browsers and devices